

Crisis Coordination Platform for Emergency Response (ResQ)

Project Proposal for UCS503P

Thapar Institute of Engineering and Technology

Group Members(3Q24):

Gurnoor Singh Marok — 1024170226

Rishi Gupta — 1024170043

Tamanna Chadha — 1027170237

Kaif Ahmad — 1024170255

1 Higher-Order Goal

This project aims to strengthen disaster and emergency-response coordination by reducing the delay between a crisis being reported and help actually reaching the affected person. While the underlying problem of coordinating independent actors under uncertainty is broader than any single deployment, the immediate engineering objective is to translate that coordination problem into a measurable, deployable software solution: a platform that connects affected people, volunteers, non-governmental organizations (NGOs), resource providers, and administrators around a shared, real-time view of who needs help and who is providing it.

2 Time-to-Value

- Meaningful increments will be delivered quickly by phasing the build into explicit sessions, each producing a working, demonstrable slice of functionality rather than a single large integrated release.
- Fast feedback will be prioritized over completeness. Phase 1, comprising request submission, registration, manual assignment, and a map view, is designed to stand on its own as a complete, demonstrable product before real-time coordination or matching logic is introduced.
- Success is defined by what can be built and self-tested within an agentic development loop (plan, code, and browser-test) in each session, and subsequently extended in the following iteration rather than attempted all at once.

3 Problem Statement

During a local emergency, the difficulty does not lie in collecting a report, which is a well-understood problem solved by a form and a map pin. The core difficulty lies in coordinating independent actors who each hold partial information and, at times, conflicting incentives: affected people, volunteers, NGOs, resource providers, and administrators. The following failure modes are commonly observed in existing approaches:

- **Duplicated or wasted effort.** Two NGOs may respond to the same request while another remains unanswered, because neither organization can observe the other's claim in real time.
- **Fragmented visibility.** Each role, namely the affected person, volunteer, NGO, and administrator, operates from an independent dashboard, with no mechanism to keep all parties synchronized as a request's status changes.
- **Unmatched partial resources.** A single need, such as transporting eight people, often requires combining several partial contributions from different providers, a scenario that conventional CRUD-based systems do not model effectively.
- **Absence of an accountability loop.** Requests are frequently marked as resolved by a responder without confirmation from the affected person that assistance was actually received, undermining trust in the system.

Contextual relevance. In a crisis, response capacity is inherently scarce. Every request that is answered twice, or that is not seen in time by the nearest available responder, represents capacity that the system has failed to utilize effectively.

4 Proposed Solution

4.1 Overview

- **Emergency reporting portal.** Affected people or bystanders submit a request with a category, description, and location; the location is made publicly visible only in an approximate, fuzzed form.
- **Verification and status workflow.** Each request progresses through a defined lifecycle rather than being represented by a single open or closed flag.
- **Cross-role real-time propagation.** When one actor accepts or updates a request, every relevant dashboard, including those of the affected person, NGO, responder, and administrator, updates automatically without requiring a manual refresh.
- **Resource and mission matching.** Volunteers, NGOs, and resource providers may be matched to requests individually, or combined into a mission when a single need requires several partial contributions.
- **Trust and accountability.** A request cannot be closed until the affected person confirms that help was actually received, and exact contact and location details are revealed only after a claim has been accepted.

4.2 Core Workflow

1. An affected person, or a bystander acting on their behalf, submits an emergency request and immediately receives a request identifier along with a confirmation.
2. The request enters the system in an Unverified state. An administrator, or an automated check, subsequently moves the request to a Verified state once basic completeness and plausibility checks are satisfied.
3. A volunteer, NGO, or resource provider claims the request, or is matched to it, at which point the request transitions to an In Progress state and is simultaneously removed from the available-request lists of other actors.
4. The responder marks the work as complete, moving the request to a Resolved state. The affected person then confirms that resolution has occurred, which is the action that formally transitions the request to a Closed state.
5. Throughout this workflow, status changes are first written to the database and subsequently broadcast over a real-time channel, ensuring that all dashboards reflect a consistent, single source of truth.

4.3 Operational Constraints for an Educational, Capstone Setting

- The technology stack will remain mainstream and well documented (React with Vite, Node.js with Express, and MongoDB with Mongoose) so that it can be built and self-tested reliably, including through the use of agentic coding tools.
- Dependence on paid infrastructure will be avoided in favor of free or self-hostable services, including MongoDB with Mongoose, Leaflet with OpenStreetMap in place of a billed maps application programming interface (API), and free-tier hosting for both the frontend and backend.

- AI-based classification will be treated as strictly optional and pluggable, such that the core coordination system continues to function if the AI module is removed or its usage quota is exhausted.

5 Solution Approach

5.1 Matching and Allocation Assistance

- Matching will be rule-based rather than learned, using a weighted score that combines location proximity, suitability, availability, and capacity, without claiming state-of-the-art performance.
- The score normalizes each factor onto a comparable scale and explicitly accounts for missing data, such as a volunteer with no declared availability window, rather than failing silently.
- Matching suggestions are presented to staff and responders for acceptance; the system does not auto-assign resources in a manner that could result in a single resource being double-booked across multiple requests simultaneously.
- A learned re-ranking model is treated as an optional, later-phase stretch goal rather than a dependency of the core workflow.

5.2 Web Architecture

- **Frontend.** A responsive React (Vite) application built with JavaScript, providing role-specific dashboards for affected persons, volunteers, NGOs, resource providers, and administrators.
- **Backend API.** A Node.js and Express service handling authentication, request submission, status transitions, and assignment and matching logic, exposed as a stateless REST API.
- **Real-time layer.** Socket.IO propagates every state change to the affected dashboards, with the database always treated as the single source of truth and sockets used purely as a push mechanism.
- **Database.** MongoDB with Mongoose as the object data modeling layer.

5.3 CI/CD and Fast Delivery Enabler

- Unit and integration tests (Vitest or Jest, and Supertest) are built and executed automatically on every change.
- Playwright end-to-end tests validate the multi-role scenarios, such as one NGO claiming a request while a second NGO's dashboard updates live, that represent the most challenging aspect of this system to implement correctly.
- Repeatable deployment artifacts are produced for a free-tier staging environment, enabling frequent, low-risk releases across the phased build sessions.

6 Evaluation Criterion

6.1 Primary Evaluation Metric

Time-to-Match (TTM): the time elapsed between a request being marked Verified and it being claimed or matched to a responder.

- Target: a median TTM of 15 minutes or less during pilot testing.
- Attribution: measured directly from database timestamps, comparing the verification time to the first assignment or claim event.

6.2 Secondary Evaluation Metrics

- **Propagation latency.** The time between a state change being written to the database and its appearance on all other affected dashboards; the target is under two seconds, with zero missed updates during pilot testing.
- **Double-booking rate.** The percentage of resources matched to more than one active request at the same time; the target is zero percent during pilot testing.
- **Confirmation rate.** The percentage of Resolved requests that reach the Closed state through affected-person confirmation, rather than being left unconfirmed.
- **Reliability.** The uptime of the staging or production environment for submission, claiming, and status updates during pilot hours, with a target of 99 percent or higher.

6.3 Pilot Validation Plan

- A small simulated pilot will be recruited, consisting of 10 to 20 users acting as affected persons or volunteers, and 3 to 5 users acting in NGO or administrator roles, should real NGO participation not be available.
- The core demonstration scenario, comprising submission, verification, claiming, cross-dashboard updating, resolution, and confirmation, will be executed end-to-end, with timestamps logged for every metric described above.
- Observed metrics will be compared against the targets defined in Sections 6.1 and 6.2, and any gaps identified will be used to prioritize the following build session.

7 Scalability

7.1 Theoretical Scalability

The system is designed to support growth in the number of concurrent requests and dashboards through the following measures:

- Decoupling the frontend and backend into independent, stateless API-driven applications;
- Indexing and paginating the queries that support request lists and mission-fulfillment views;
- Keeping the database, rather than the socket layer, as the single source of truth, so that horizontal scaling of the real-time layer does not risk inconsistent state.

7.2 Practical Deployability

The system is designed to run entirely on free-tier infrastructure through the following measures:

- Using MongoDB Atlas, or another suitable free-tier MongoDB deployment option;
- Deploying the frontend and backend independently on free-tier platforms, such as Vercel or Netlify for the frontend and Render or Railway for the backend;

- Running the AI classification module as a separable microservice that can be scaled, disabled, or replaced without modifying the core system.

8 Engine Availability Heuristic

A mature set of tools and services is available to implement this project as a web-based, real-time platform without the need to build the following components from scratch:

- A web framework for REST APIs and a componentized frontend, namely Express and React with Vite;
- A document database with a mature object data modeling layer, namely MongoDB with Mongoose;
- A real-time transport mechanism for cross-dashboard propagation, namely Socket.IO;
- An authentication approach for five distinct roles, implemented with JSON Web Tokens (JWT) and bcrypt, requiring no paid authentication vendor;
- A mapping and geocoding solution that avoids billed APIs, namely Leaflet, OpenStreetMap, and Nominatim;
- Object storage for optional attachments, using local storage during the pilot and Cloudflare R2 or Supabase Storage free tiers for deployment;
- Test frameworks for unit, integration, and end-to-end coverage, namely Vitest or Jest, Supertest, and Playwright;
- Optional large language model (LLM) access for free-text urgency and category classification, maintained as a separate, removable service.

These mature, mainstream components are used deliberately, so that effort is directed toward coordination correctness and workflow design rather than toward building infrastructure that already exists.

9 Project Scope and Deliverables

9.1 Initial Deliverable

- A request submission form with immediate request identifier generation, and location captured with a distinction between public and precise representations;
- Registration for volunteers, NGOs, and resource providers, with role-based access control;
- Manual administrator assignment and the full status lifecycle, consisting of the Unverified, Verified, In Progress, Resolved, and Closed states;
- A Leaflet-based map view of open requests;
- Continuous integration, comprising build, backend unit tests, and linting, together with continuous deployment to a single free-tier staging environment.

9.2 Subsequent Deliverables

- **Phase 1, Coordination.** Socket.IO integration for the four-dashboard real-time propagation scenario, resource allocation tables, mission management, in-app messaging, and the resolution and confirmation flow.
- **Phase 2, Advanced Features.** The rule-based matching engine described in Section 5.1, an analytics and administration dashboard, and the optional AI classification microservice.
- **Phase 3, Hardening and Demonstration.** Playwright end-to-end scripts for the complete multi-role scenario, representative seed data, and deployment to free-tier hosting for the live submission.

10 Risks and Mitigations

- **Scope creep due to entity count.** Five roles, multiple dashboards, missions, matching, and optional AI functionality together represent substantial surface area. This risk is mitigated by treating the phase gating described in Section 9 as load-bearing rather than decorative, and by ensuring that Phase 1 constitutes a complete product on its own.
- **Underestimated complexity of the matching algorithm.** Normalizing distance against capacity, handling missing availability data, and preventing a single resource from being suggested to multiple requests simultaneously all require explicit design consideration. This risk is mitigated through database-level transaction locks around the claim and assignment process.
- **Real-time correctness exceeding real-time plumbing.** Every state mutation must follow a strict discipline of writing to the database first and broadcasting second. This risk is mitigated by centralizing state transitions through a single code path rather than duplicating broadcast logic across handlers.
- **Location and contact privacy.** Details pertaining to an affected person cannot be broadcast to every volunteer. This risk is mitigated by displaying only an approximate location publicly and revealing the precise location and contact information only after a claim has been accepted.
- **Complexity of mission-fulfilment tracking.** Reliably summing partial allocations against a permission, per-category target requires an explicit quantity-fulfilled schema that is decided before development begins, rather than retrofitted afterward.

11 Summary

The implementation will use the MERN stack, comprising MongoDB, Express.js, React.js, and Node.js, supplemented by Socket.IO, Leaflet with OpenStreetMap, JWT with bcrypt, and free or open-source and free-tier services to keep the project cost-effective and suitable for an educational deployment.

This project proposes a deployable, real-time coordination platform to reduce the time between an emergency being reported and help actually being delivered in a resource-constrained response setting. It satisfies the course criteria by emphasizing time-to-value through explicitly phased build sessions, by using continuous integration and continuous deployment to support frequent and safe releases, and by defining measurable evaluation criteria, principally time-to-match and cross-dashboard propagation latency. The proposal remains focused on engineering coordination correctness, real-time consistency, and scalability readiness, while treating AI-assisted classification as an optional, decoupled enhancement rather than a core dependency.